Practical – 13

Aim – 1 : Generic containers

A generic Box with get/set and a Pair; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

Code :

Main.java

```java
class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

class Pair<A, B> {
    private A first;
    private B second;

    public Pair(A first, B second) {
        this.first = first;
        this.second = second;
    }

    public A getFirst() {
        return first;
    }

    public B getSecond() {
        return second;
    }
}
```

ady >_ Terminal

Main.java

```java
public class GenericContainers {
    // Generic method to return first element of any array
    public static <T> T firstElement(T[] array) {
        return array[0];
    }

    public static void main(String[] args) {

        // Box with String
        Box<String> nameBox = new Box<>();
        nameBox.set("Kavya");
        System.out.println("Box String: " + nameBox.get());

        // Box with Integer
        Box<Integer> numberBox = new Box<>();
        numberBox.set(100);
        System.out.println("Box Integer: " + numberBox.get());

        // Pair with String and Integer
        Pair<String, Integer> student =
                new Pair<>("Kavya", 25);

        System.out.println("Pair First: " + student.getFirst());
        System.out.println("Pair Second: " + student.getSecond());

        // Generic method
        String[] names = {"Kavya", "Rahul", "Amit"};
        Integer[] numbers = {10, 20, 30};

        System.out.println("First name: " + firstElement(names));
```

dy >_ Terminal

Output :

```
Box String: Kavya
Box Integer: 100
Pair First: Kavya
Pair Second: 25
First name: Kavya
First number: 10
```

Learning Outcome

- Understand **generic classes** using `<T>`.
- Use `Box<T>` with different data types.
- Use multiple type parameters with `Pair<A,B>`.
- Understand **type safety without explicit type casting**.
- Create and use **generic methods**.

Aim – 2 : . Bounded stats

a generic method double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

Code :

Main.java

```java
public class BoundedStats {

    // Generic method restricted to Number types
    public static <N extends Number> double average(N[] values) {

        double sum = 0;

        for (N value : values) {
            sum += value.doubleValue();
        }

        return sum / values.length;
    }

    public static void main(String[] args) {

        Integer[] integers = {10, 20, 30, 40};
        Double[] doubles = {2.5, 3.5, 4.5};
        Long[] longs = {100L, 200L, 300L};

        System.out.println("Integer average: " +
                average(integers));

        System.out.println("Double average: " +
                average(doubles));

        System.out.println("Long average: " +
                average(longs));
    }
}
```

Terminal

Output :

Console | I/O | AI Agent | 981 ms

```
Integer average: 25.0
Double average: 3.5
Long average: 200.0
```

Learning Outcome

- Understand **bounded generics**.
- Use `<N extends Number>` to restrict acceptable types.
- Use `doubleValue()` to perform calculations on different numeric types.
- Understand **compile-time type safety**.
- Learn why non-numeric types such as `String` are rejected.

Aim – 3 : For Advanced Learners – home

Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

Code :

Main.java

```java
class GenericStack<T> {

    private T[] stack;
    private int top;

    @SuppressWarnings("unchecked")
    public GenericStack(int size) {
        stack = (T[]) new Object[size];
        top = -1;
    }

    // Push element
    public void push(T value) {

        if (top == stack.length - 1) {
            System.out.println("Stack Overflow");
            return;
        }

        stack[++top] = value;
    }

    // Pop element
    public T pop() {

        if (top == -1) {
            System.out.println("Stack is empty");
            return null;
        }
```

eady >_ Terminal


Main.java
class GenericStack<T> {
public T pop() {
return stack[top--];
}
// Display stack
public void display() {
if (top == -1) {
System.out.println("Stack is empty");
return;
}
for (int i = top; i >= 0; i--) {
System.out.println(stack[i]);
}
}
}
public class GenericStackDemo {
public static void main(String[] args) {
// Stack of Integer
GenericStack<Integer> numbers = new GenericStack<>(5);
numbers.push(10);
numbers.push(20);
numbers.push(30);
Terminal

Main.java

```
public class GenericStackDemo {
    public static void main(String[] args) {
        numbers.push(10);
        numbers.push(20);
        numbers.push(30);

        System.out.println("Integer Stack:");
        numbers.display();

        System.out.println("Popped: " + numbers.pop());

        // Stack of String
        GenericStack<String> names = new GenericStack<>(5);

        names.push("Kavya");
        names.push("Rahul");
        names.push("Amit");

        System.out.println("\nString Stack:");
        names.display();

        System.out.println("Popped: " + names.pop());

        // Empty case
        GenericStack<Double> emptyStack = new GenericStack<>(3);

        System.out.println("\nTrying to pop from empty stack:");
        System.out.println("Popped: " + emptyStack.pop());
    }
}
```

Terminal

Output :

```
Integer Stack:
30
20
10
Popped: 30

String Stack:
Amit
Rahul
Kavya
Popped: Amit

Trying to pop from empty stack:
Stack is empty
Popped: null
```

Learning Outcome – Problem 3

- Understand how to create a **generic stack using an array**.
- Learn to reuse the same stack for **different data types**.
- Understand **LIFO (Last In, First Out)** operations.
- Handle **stack overflow and empty stack** conditions.
- Apply **generics for type safety and code reusability**.